



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Instituto Tecnológico de Tepic

Ingeniería en sistemas computacionales

Programación Web

Actividad 1: Cuadro sinóptico

Facilitador: Nava Hernández Irving Yair

Alumna: Cervantes Pérez Valeria

Número de control: 23400522

Séptimo semestre

Tepic, Nayarit, 13 de julio de 2026

JavaScript

JavaScript es un lenguaje de programación interpretado, ligero y de tipado débil. Se ejecuta principalmente en el navegador web para crear contenido dinámico e interactivo, aunque también se utiliza del lado del servidor y en múltiples plataformas mediante entornos como Node.js.

Entre sus características fundamentales destacan:

Interpretado y JIT (Just-In-Time): El código no requiere ser compilado antes de ejecutarse. El navegador lee, interpreta y ejecuta el código en tiempo real, aunque utiliza compilación justo-a-tiempo para optimizar la velocidad.

Multiparadigma: Soporta programación orientada a objetos (basada en prototipos), imperativa y funcional.

Tipado débil y dinámico: No es necesario declarar el tipo de dato de una variable al crearla y una misma variable puede contener diferentes tipos de datos (números, textos, objetos) a lo largo del programa.

Asíncrono y basado en eventos: Permite ejecutar operaciones simultáneas en segundo plano (como consultar una base de datos o descargar un archivo) sin bloquear el hilo principal de la interfaz de usuario.

Multiplataforma y versátil: Es compatible con todos los navegadores web modernos y sirve tanto para el desarrollo front-end como para el back-end.

Condicionales

Permiten ejecutar diferentes bloques de código basándose en si una expresión es verdadera o falsa.

- **if / else if / else:** Ejecuta un bloque si la condición inicial se cumple, de lo contrario evalúa otras alternativas o ejecuta un bloque por defecto.
- **Operador ternario:** Un atajo para escribir un if de una sola línea: `condicion ? valorSiVerdadero : valorSiFalso`.
- **switch:** Evalúa una expresión y compara su valor contra múltiples etiquetas (case). Es útil para evitar largas cadenas de if...else.

Bucles o Ciclos (Loops)

Se utilizan para repetir una porción de código mientras una condición específica se mantenga.

- **for:** Repite un bloque un número determinado de veces. Ideal cuando conoces el número exacto de iteraciones por adelantado.

- **while:** Ejecuta un bloque de código mientras la condición evaluada sea verdadera.
- **do...while:** Similar a while, pero garantiza que el bloque de código se ejecutará al menos una vez antes de verificar la condición.

Tipos de datos

JavaScript tiene 8 tipos de datos básicos, divididos en dos categorías principales: primitivos y objetos. Son dinámicos, lo que significa que una variable no está ligada a un solo tipo.

Tipos Primitivos

Son los bloques de construcción más simples:

- **Undefined** (undefined): Representa una variable declarada a la que aún no se le asigna ningún valor.
- **Null** (null): Representa una ausencia intencional de cualquier valor o objeto.
- **Boolean** (true o false): Valores lógicos que representan condiciones verdaderas o falsas.
- **Number:** Representa números enteros y de punto flotante (decimales).
- **BigInt:** Permite representar de forma segura números enteros excesivamente grandes que superan el límite del tipo *Number*.
- **String:** Cadenas de texto que van entre comillas simples o dobles.
- **Symbol:** Valores únicos e inmutables, frecuentemente utilizados para identificar propiedades de objetos de forma exclusiva.

Tipos de Objetos

- **Object** (object): Estructuras más complejas que permiten almacenar colecciones de datos y entidades más elaboradas. Incluye funciones, arreglos (arrays) y fechas.

Funciones:

Las funciones en JavaScript son bloques de código reutilizables diseñados para realizar una tarea específica. Se definen usando la palabra clave `function` o mediante sintaxis de flecha, pueden recibir parámetros como entrada y utilizan `return` para devolver un resultado.

Declaración de función (Function Declaration)

Es la forma tradicional de definir una función. Tienen la ventaja del *hoisting*, lo que significa que puedes llamarlas antes de haberlas escrito en el código.

Expresión de función (Function Expression)

Consiste en almacenar una función dentro de una variable. Estas funciones no se pueden llamar antes de ser declaradas.

Funciones Flecha (Arrow Functions)

Introducidas en ECMAScript 6, ofrecen una sintaxis más corta y limpia. Si solo tienen una instrucción, pueden omitir las llaves y el return.

Parámetros y Retorno

- **Parámetros:** Son las variables que la función espera recibir entre los paréntesis. Puedes asignarles valores por defecto.
- **Retorno (return):** Es la salida de la función; detiene la ejecución y devuelve el resultado.

Objetos y arrays

Los arrays son listas ordenadas de datos que utilizan índices numéricos (desde 0), ideales para colecciones. Los objetos almacenan datos en pares clave-valor y se usan para agrupar características de una entidad. Combinarlos creando "arrays de objetos" es la estructura más común para manejar datos complejos.

Arrays (Arreglos)

Colección ordenada de elementos. Se definen entre corchetes [].

Objetos

Colección de propiedades (claves y valores). Se definen entre llaves {}.

Arrays de Objetos

La estructura más potente y utilizada para modelar bases de datos o listas de entidades.

Métodos principales y herramientas

Para manipular estas estructuras, JavaScript provee funciones nativas muy eficientes.

- **map():** Crea un nuevo array transformando cada elemento.
- **filter():** Filtra elementos basándose en una condición.
- **find():** Busca y retorna el primer elemento que cumpla con la condición.

Eventos y manipulación del DOM

La manipulación del DOM y los eventos en JavaScript permiten crear páginas web dinámicas. Puedes usar `querySelector` o `getElementById` para seleccionar

elementos, modificarlos con propiedades como `.textContent` o `.innerHTML`, y hacerlos interactivos agregando detectores con `addEventListener` para escuchar acciones como clics o envíos.

1. Selección y Modificación de Elementos

Para interactuar con el HTML, primero debes seleccionar el nodo mediante métodos del objeto `document`:

- `document.getElementById('id')`: Selecciona un elemento específico por su ID.
- `document.querySelector('.clase')`: Selecciona el primer elemento que coincida con el selector CSS.
- `document.querySelectorAll('etiqueta')`: Selecciona todos los elementos que coincidan y devuelve una lista.

2. Manejo de eventos (Event Listeners)

Los eventos son las acciones que ocurren en la página (como clics, movimientos del ratón o teclas presionadas). El método `addEventListener()` permite adjuntar una función que se ejecutará únicamente cuando el evento ocurra.

Los eventos más comunes en la web son:

- `click`: Ocurre cuando el usuario hace clic en un elemento.
- `input / change`: Se activan al escribir o modificar un campo de entrada.
- `submit`: Fundamental para validar y controlar el envío de formularios.
- `keydown / keyup`: Detectan cuando el usuario presiona o suelta una tecla.

3. Crear y Eliminar Elementos

JavaScript también permite alterar la estructura HTML existente:

- `document.createElement('etiqueta')`: Crea un nuevo elemento en memoria.
- `elementoContenedor.appendChild(nuevoElemento)`: Agrega el elemento dentro del contenedor especificado.
- `elemento.remove()`: Elimina el elemento directamente del DOM.

Asincronía

La asincronía en JavaScript es la capacidad del lenguaje para delegar tareas de larga duración (como peticiones HTTP o temporizadores) a segundo plano, permitiendo que el hilo principal siga ejecutando código sin congelar la interfaz de usuario. Aunque JavaScript es de un solo hilo (single-threaded), logra este

comportamiento no bloqueante apoyándose en el entorno de ejecución (navegador o Node.js) y en su famoso Event Loop.

1. Callbacks (*Funciones de retorno*)

Es el método tradicional. Consiste en pasar una función como argumento a otra para que se ejecute cuando la tarea finalice.

- **Desventaja:** Provoca el *Callback Hell* (código anidado ilegible en forma de pirámide).

2. Promesas (*Promises*)

Introducidas en ES6, representan el **resultado futuro** (exitoso o fallido) de una operación asíncrona. Tienen tres estados: *Pending* (Pendiente), *Fulfilled* (Resuelta) y *Rejected* (Rechazada).

- **Ventaja:** Permiten encadenar métodos `.then()` y capturar errores globalmente con `.catch()`.

3. Async / Await

Introducido en ES8, es **azúcar sintáctico** estructurado sobre las promesas. Permite escribir código asíncrono que se lee de forma secuencial y limpia, como si fuera síncrono.

- **Requisito:** `await` solo se puede usar dentro de funciones marcadas con `async`.
- **Manejo de errores:** Se realiza mediante bloques estándar `try / catch`.

Cuadro sinóptico

